令和８年度 情報工学実験III 報告書

実験題目

プログラミング演習III

指導教員

滝沢教員

実験日　令和８年度　９月２５日（金）
令和８年度１０月０２日（金）

レポート
提出締切日　令和８年度　１０月　９日（金）

レポート
受理最終日　令和８年度　１０月１６日（金）

レポート
提出日　令和８年度　９月２５日（金）

報告者　橋本 千聡

実験の目的

既存プログラムへの新規機能の追加。

- 実用的なプログラム/ソフトウェアの開発においては,使い捨てではなく,既存のソースコードや開発文書を再利用して
- 機能を追加したり不具合を解消したりするのは,必ずしも初期開発に携わった者とは限りません.
- 開発(保守)を継続するには,既存のソースコードや開発文書を理解・解析する必要があります.
- 本実験では,プログラム/ソフトウェアの保守過程を疑似的に体験し,対応するコードや文書を仕上げて提出してもらい
- 併せて,理解・解析の補助ツールとして,AI chatbot(ChatGPT,Gemini,Copilot,等)の適切な活用を学びます.

課題 1

起動方法と終了方法

- nanoなどと同様に、最初の引数のファイルを開くことができる。
- ccでコンパイルした場合、`./a.out <ファイル名>`で起動できる。
- 終了時は `Ctrl + Q` で終了できる。
- `Ctrl + C` や `Ctrl + D`、 `Ctrl + \` などでは強制終了できない。

スクリーン移動方法

- 上下左右矢印キーでカーソルを一文字づつ移動することができる。
- `PgUp`、`PgDn` でスクロール(＝ページ単位)での移動ができる。

課題 2

機能	入/出	エスケープシーケンス	説明
カーソルを上へ移動	入力	`ESC[A`	上へ移動
カーソルを下へ移動	入力	`ESC[B`	下へ移動
カーソルを右へ移動	入力	`ESC[C`	右へ移動
カーソルを左へ移動	入力	`ESC[D`	左へ移動
カーソルをホームへ移動	入力	`ESC[H` または `ESC0H`	Homeキー
カーソルを行末へ移動	入力	`ESC[F` または `ESC0F`	Endキー
Deleteキー	入力	`ESC[3~`	Deleteキー
PageUpキー	入力	`ESC[5~`	PageUpキー
PageDownキー	入力	`ESC[6~`	PageDownキー
カーソルを右端へ移動	出力	`ESC[999C`	右端へ移動
カーソルを下端へ移動	出力	`ESC[999B`	下端へ移動
カーソル位置の問い合わせ	出力	`ESC[6n`	位置を問い合わせ
カーソル位置の報告	入力	`ESC[n;mR`	n行m列を報告

課題 3

```
struct editorConfig {
    int cx,cy;  /* Cursor x and y position in characters */
    int rowoff;     /* Offset of row displayed. */
    int coloff;     /* Offset of column displayed. */
    int screenrows; /* Number of rows that we can show */
    int screencols; /* Number of cols that we can show */
    int numrows;    /* Number of rows */
    int rawmode;    /* Is terminal raw mode enabled? */
    erow *row;      /* Rows */
    int dirty;      /* File modified but not saved. */
    char *filename; /* Currently open filename */
    char statusmsg[80];
    time_t statusmsg_time;
};
```

cx, cy, rowoff, coloff

- `cx`, `cy` は現在表示している画面内でのカーソルの横位置と縦位置を表す。0から始まる値で、端末上の列番号とは1ず
- `rowoff`, `coloff` は画面の最上段に表示しているファイル行の番号と、画面の左端に表示しているファイル上の文字位
- `rowoff` はカーソルが画面の最上段や最下段を越えて移動すると、この値を変更して表示範囲を上下へスクロールする

- rowoffが0のとき、画面の最上段にファイルの先頭行が表示されていることを意味する。
- coloffは長い行の右側を表示するときに増加し、左側へ戻るときに減少する。
- coloffが0のとき、画面の左端にファイルの先頭文字が表示されていることを意味する。
- 実際のファイル上の列位置を求めるときは、coloff + cx、行番号を求めるときは、rowoff + cyを使う。

cx, cyは画面内の相対位置を表し、rowoff, coloffはファイル内の表示開始位置を表す。

screenrows, screencols, numrows, row

- screenrowsはエディタの本文に利用できる行数で、端末の行数から下2行分を引いた値になる。
- screencolsはエディタの本文に利用できる列数で、端末の列数と一致する。
- numrowsは現在開いているファイルの行数で、ファイルを読み込むとき、editorInsertRow()が行を1つ追加するたびに
- rowはファイルの各行を格納するerow構造体の配列へのポインタで、E.row[i]がファイルのi行目に対応する。

rawmode, dirty, filename

- rawmodeは端末がRawモードになっているかを示す値である。Rawモードでは、入力されたキーをEnterを押すまで待
- dirtyはファイルが変更されたかを表す値である。0なら保存後または未変更、0以外なら変更ありとして扱われる。
  Qを押したときに警告を出したりするために使われる。
- filenameは現在開いているファイル名へのポインタである。ステータスバーにファイル名を表示するときや、ファイ

statusmsg[80], statusmsg_time

- statusmsg[80]は、画面最下行に表示する状態メッセージを格納する文字配列である。警告や操作結果などを表示する
- statusmsg_timeは、状態メッセージを設定した時刻である。editorRefreshScreen()では、メッセージが設定されてか

editorRefreshScreen()での動的な使われ方

editorRefreshScreen()は、struct editorConfig Eに保存された状態を読み取り、現在のエディタ画面を端末へ描画する関数

まず、E.rowoffを基準にして、画面の各行に対応するファイル上の行番号をE.rowoff +
yで求める。求めた行番号がE.numrows未満なら、E.rowから該当するerowを取り出して表示する。ファイルの行数を超えた

各行の表示ではE.coloffを使い、ファイル上のどの列から表示するかを決める。これにより、画面幅を超える長い行でも

ステータスバーでは、E.filenameからファイル名を、E.numrowsからファイルの行数を取得する。E.dirtyが0以外の場合は、
+ E.cy + 1を使い、現在表示している行番号を表示する。

画面最下行には、E.statusmsgとE.statusmsg_timeを使って、設定されてから5秒未満の状態メッセージだけを表示する。最

このように、キー入力によってE.cx、E.cy、E.rowoff、E.coloffなどが変更されると、次回のeditorRefreshScreen()が変更

課題 4

8班なので -で一行前の先頭へ、Enterで一行後の先頭へ移動する機能を追加した。

-を押されたらARROW_UPと同様の処理のあと、カーソルの横位置を0にするため、cxとcoloffを0に設定する。

Enterも同様にARROW_DOWNの処理後にcx, coloffを0に設定する。

詳細設計

処理の流れ

```
flowchart TD
    A[キー入力を読み取る] --> B{入力は - か}
    B -- Yes --> C[editorMoveCursor ARROW_UP]
    C --> D[cx = 0<br/>coloff = 0]
    B -- No --> E{入力はEnterか}
    E -- Yes --> F[editorMoveCursor ARROW_DOWN]
    F --> G[cx = 0<br/>coloff = 0]
    E -- No --> H[既存のキー処理]
    D --> I[画面を再描画]
```

```
    G --> I
    H --> I
```

-の場合は既存の上方向移動処理を利用し、Enterの場合は既存の下方向移動処理を利用する。editorMoveCursor()が画面端

実装

editorProcessKeypress()のキー判定に、次の2つの処理を追加した。

```
case '-':
    /* Move to the beginning of the previous line. */
    editorMoveCursor(ARROW_UP);
    E.cx = 0;
    E.coloff = 0;
    break;
case ENTER:
    /* Move to the beginning of the next line. */
    editorMoveCursor(ARROW_DOWN);
    E.cx = 0;
    E.coloff = 0;
    break;
```

ENTERはenum KEY_ACTIONですでに値13として定義されているため、キー定義の追加は必要ない。-は通常の文字入力としてeditorReadKey()から返されるため、editorProcessKeypress()のswitch文で直接判定する。

テスト

次の項目を確認する。

テスト項目	操作	期待する結果
1行前への移動	2行目以降でカーソルを途中まで移動し、-を押す	1行前の先頭へ移動する
1行後への移動	先頭行以外でEnterを押す	1行後の先頭へ移動する
横スクロールの解除	長い行を右側まで移動してから-またはEnterを押す	移動先の行が左端から表示され、カーソルが行頭にある
ファイル先頭の境界	1行目で-を押す	先頭より上へ移動しない
ファイル末尾の境界	最終行でEnterを押す	末尾より下へ移動しない
画面スクロール	画面最上段または最下段付近で-・Enterを押す	必要に応じて画面が上下にスクロールする

コンパイル時には、kilov.cをコンパイルしてエラーが発生しないことを確認する。実行時には複数行のテキストファイル

テスト画面

いかが git diffで出力された差分である。

```
void editorProcessKeypress(int fd) {
    /* When the file is modified, requires Ctrl-q to be pressed N times
     * before actually quitting. */
    static int quit_times = KILO_QUIT_TIMES;

    int c = editorReadKey(fd);
    switch(c) {
    case CTRL_C:        /* Ctrl-c */
        /* We ignore ctrl-c, it can't be so simple to lose the changes
         * to the edited file. */
        break;
    case CTRL_Q:        /* Ctrl-q */
        /* Quit if the file was already saved. */
        if (E.dirty && quit_times) {
            editorSetStatusMessage("WARNING!!! File has unsaved changes. "
                "Press Ctrl-Q %d more times to quit.", quit_times);
            quit_times--;
            return;
        }
```

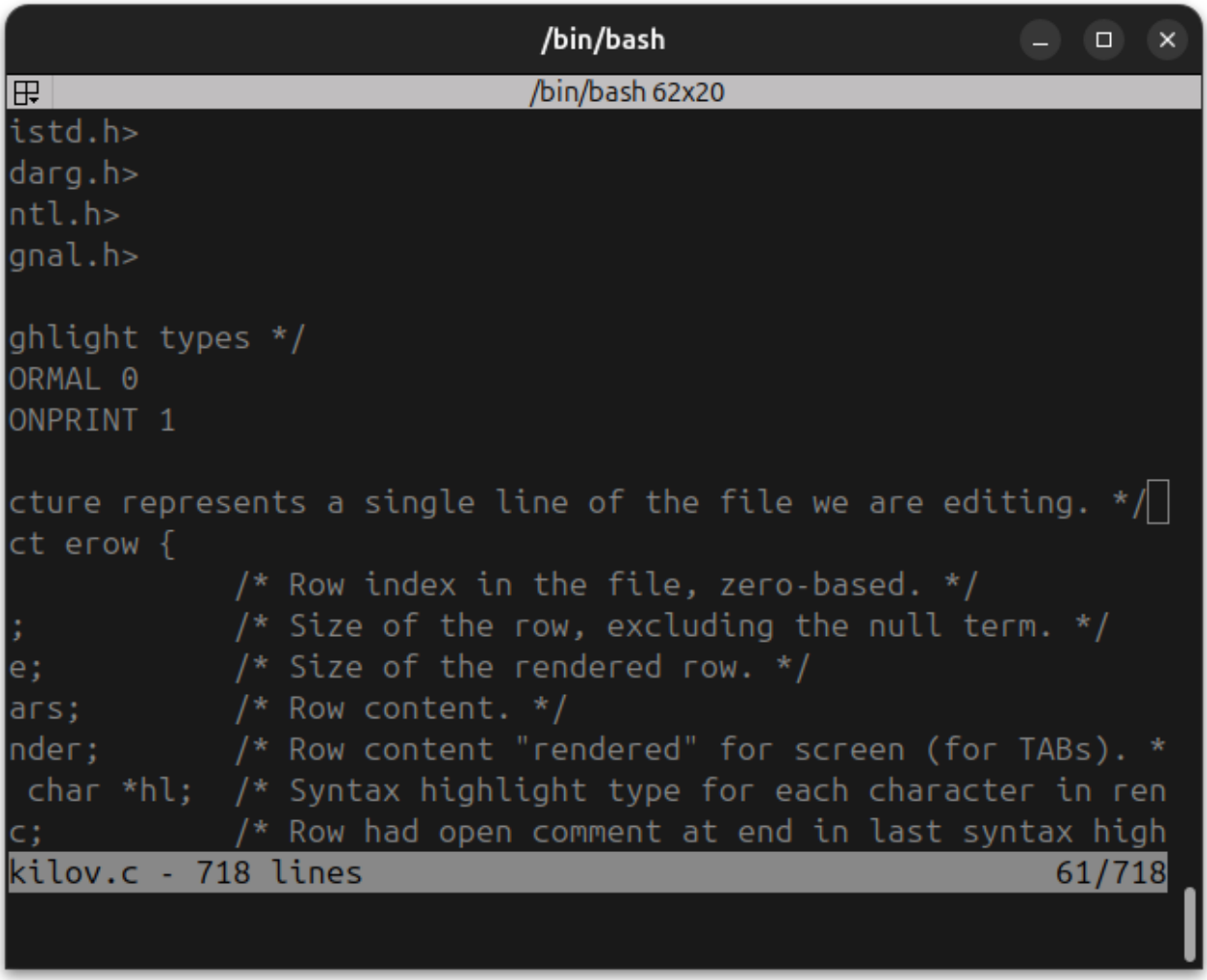


Figure 1: 課題4テスト1（-を押す前）

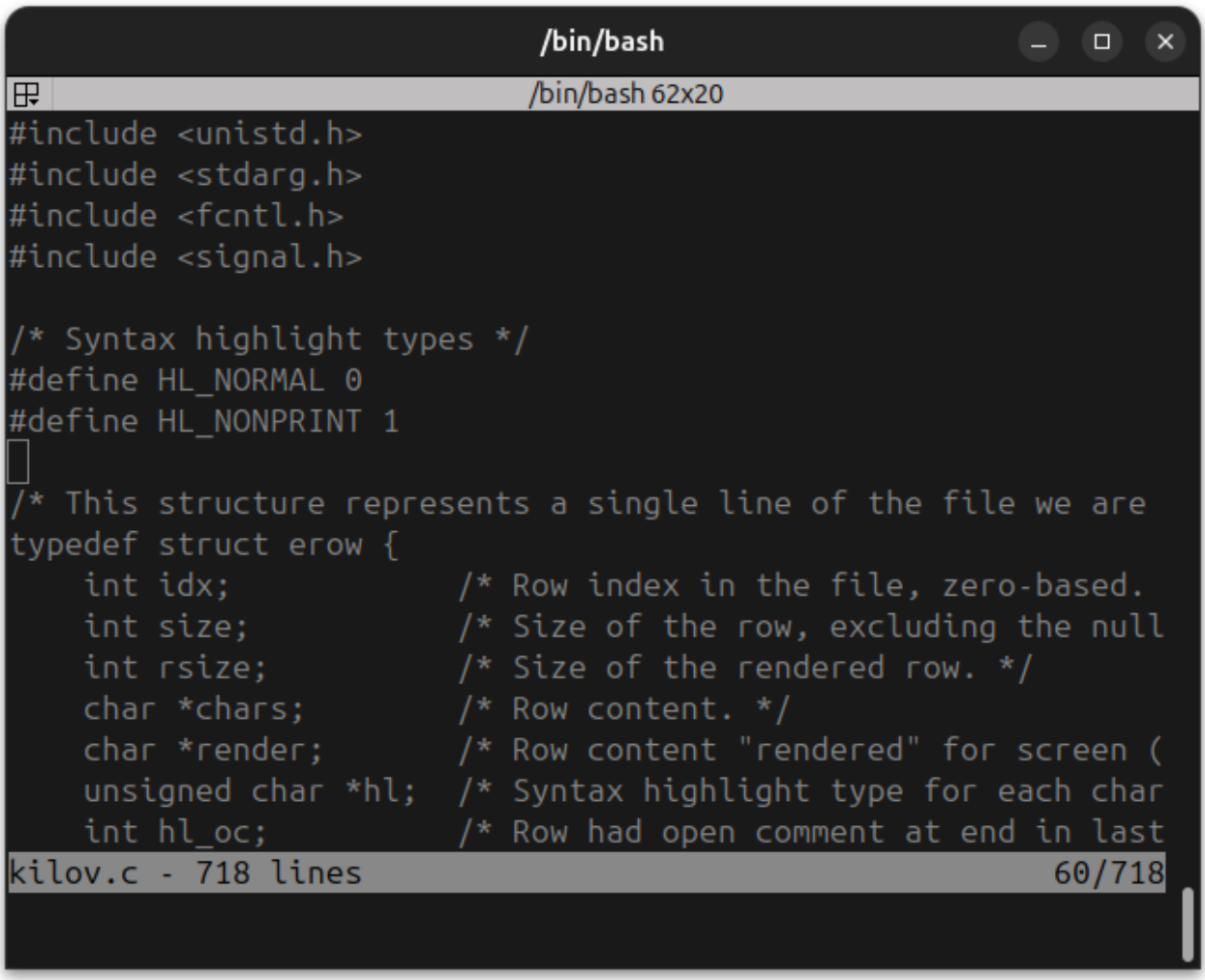


Figure 2: 課題4テスト2（-を押した後）

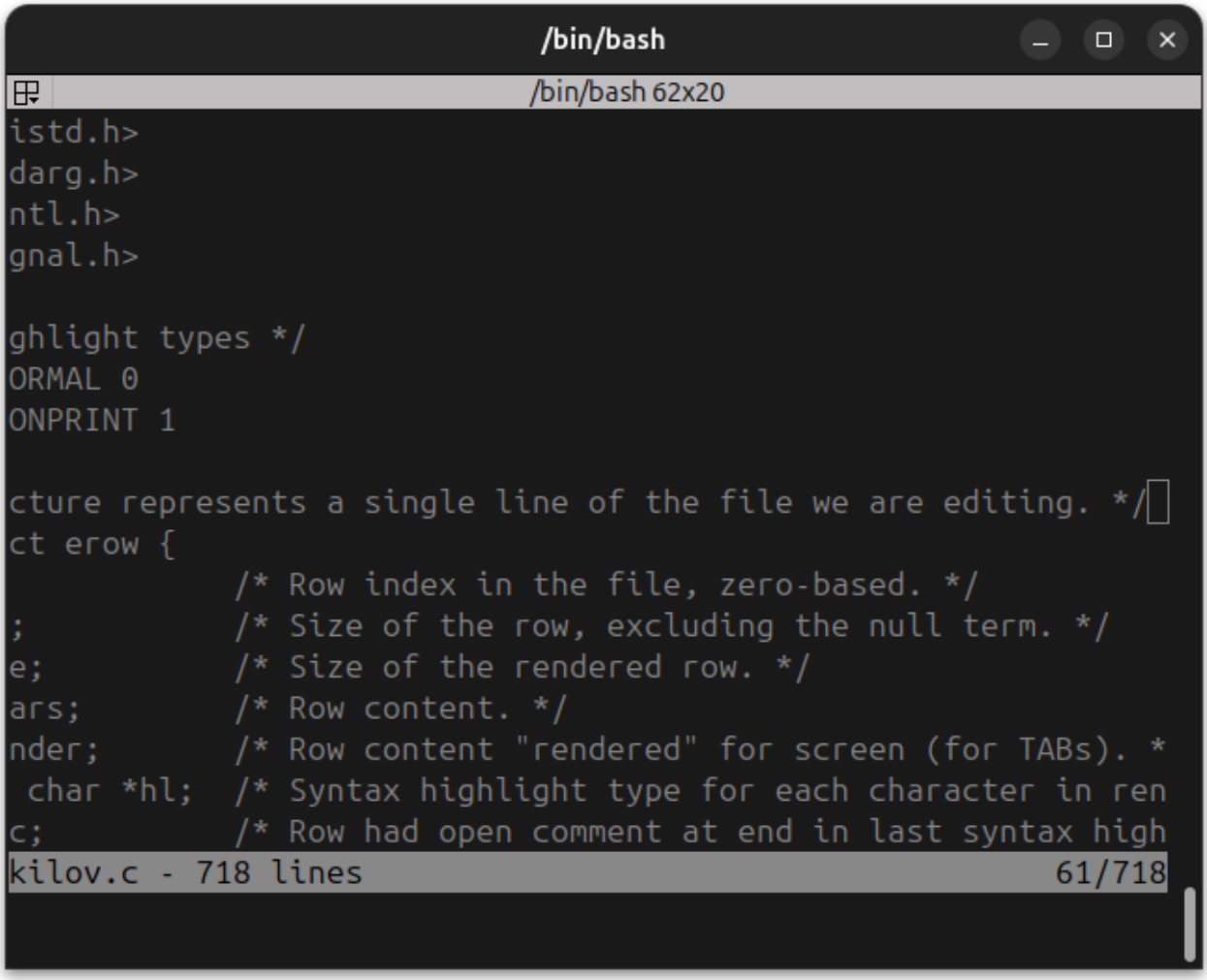


Figure 3: 課題4テスト3（Enterを押す前）

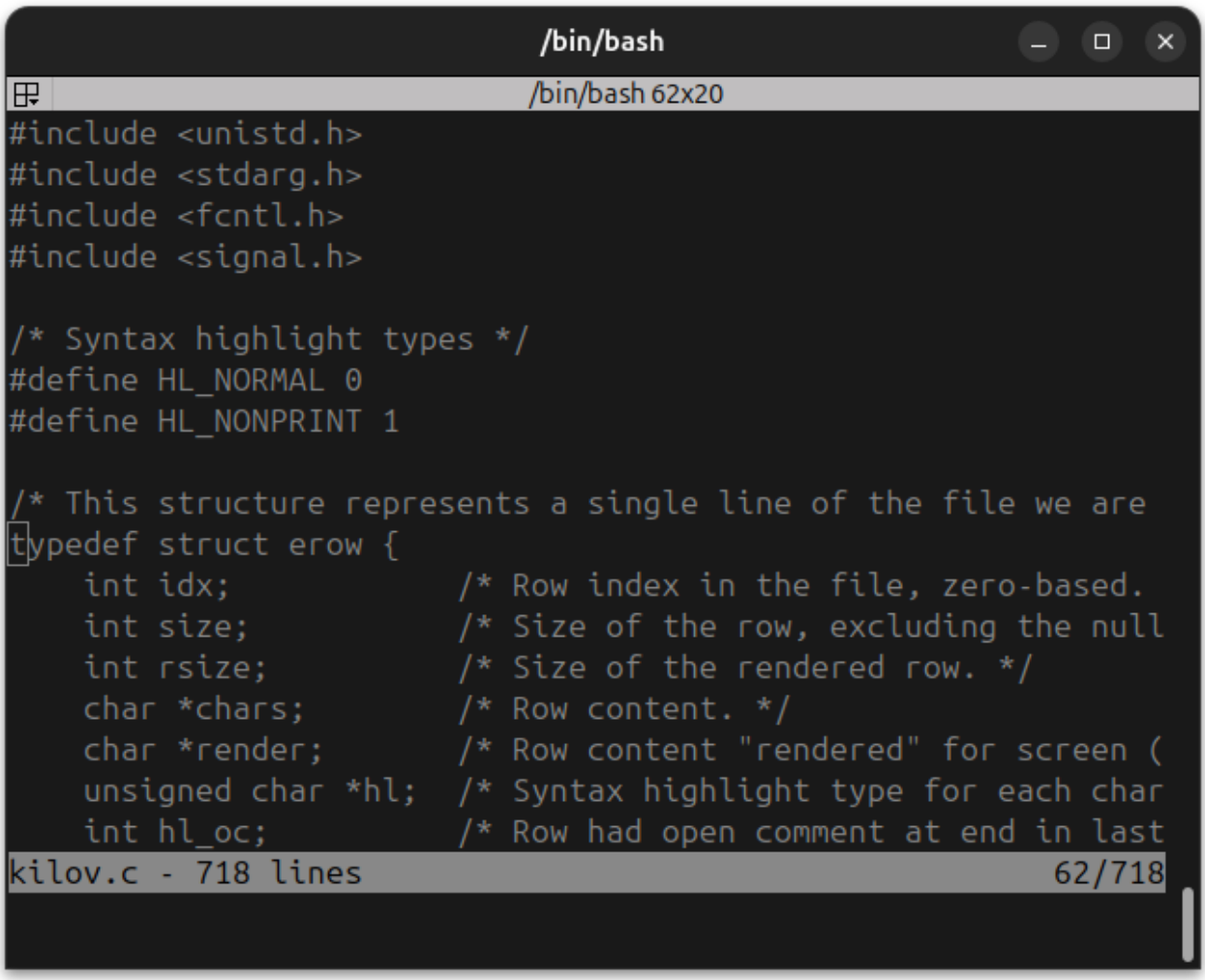


Figure 4: 課題4テスト4（Enterを押した後）

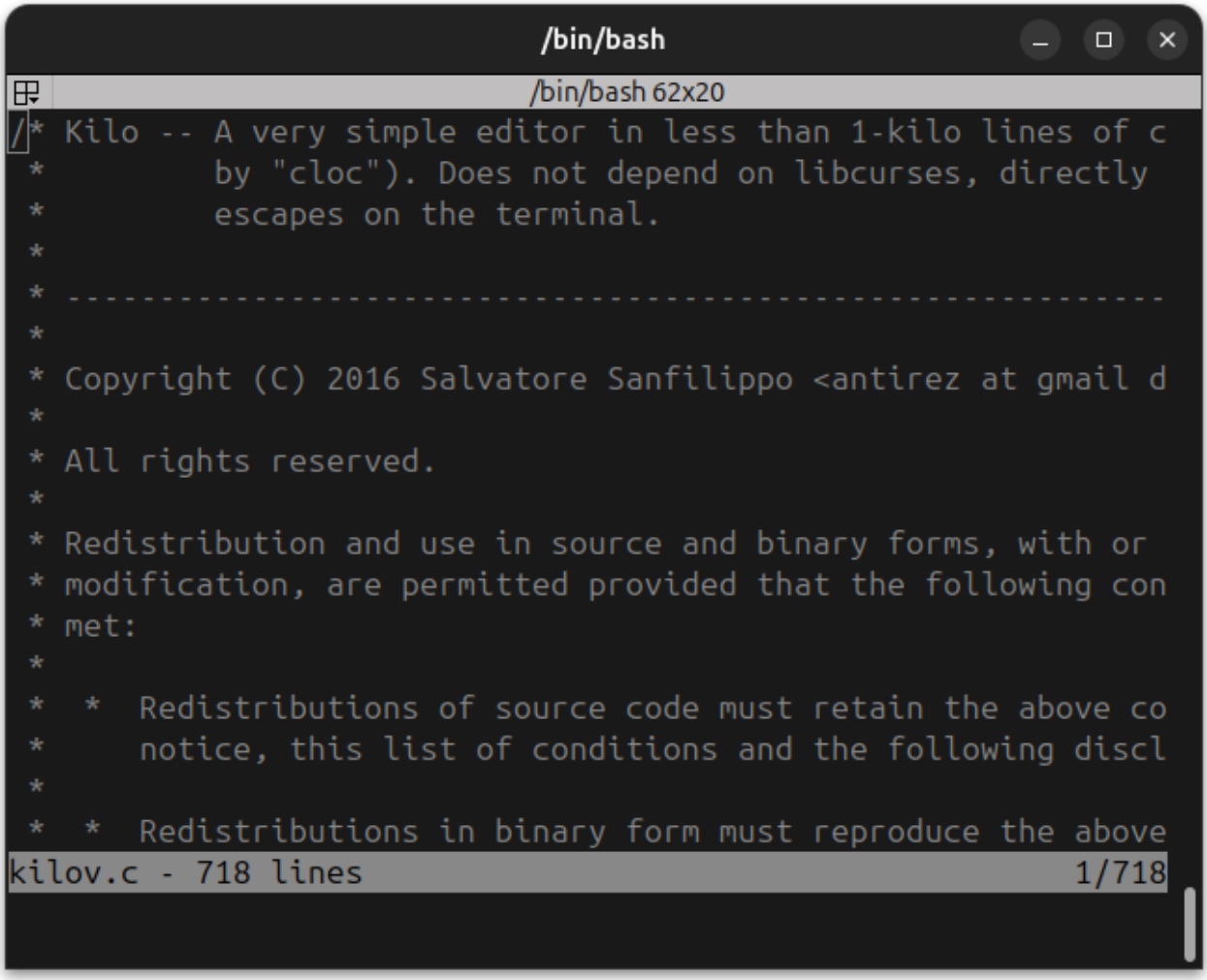


Figure 5: 課題4テスト5（1行目で-を押した後）

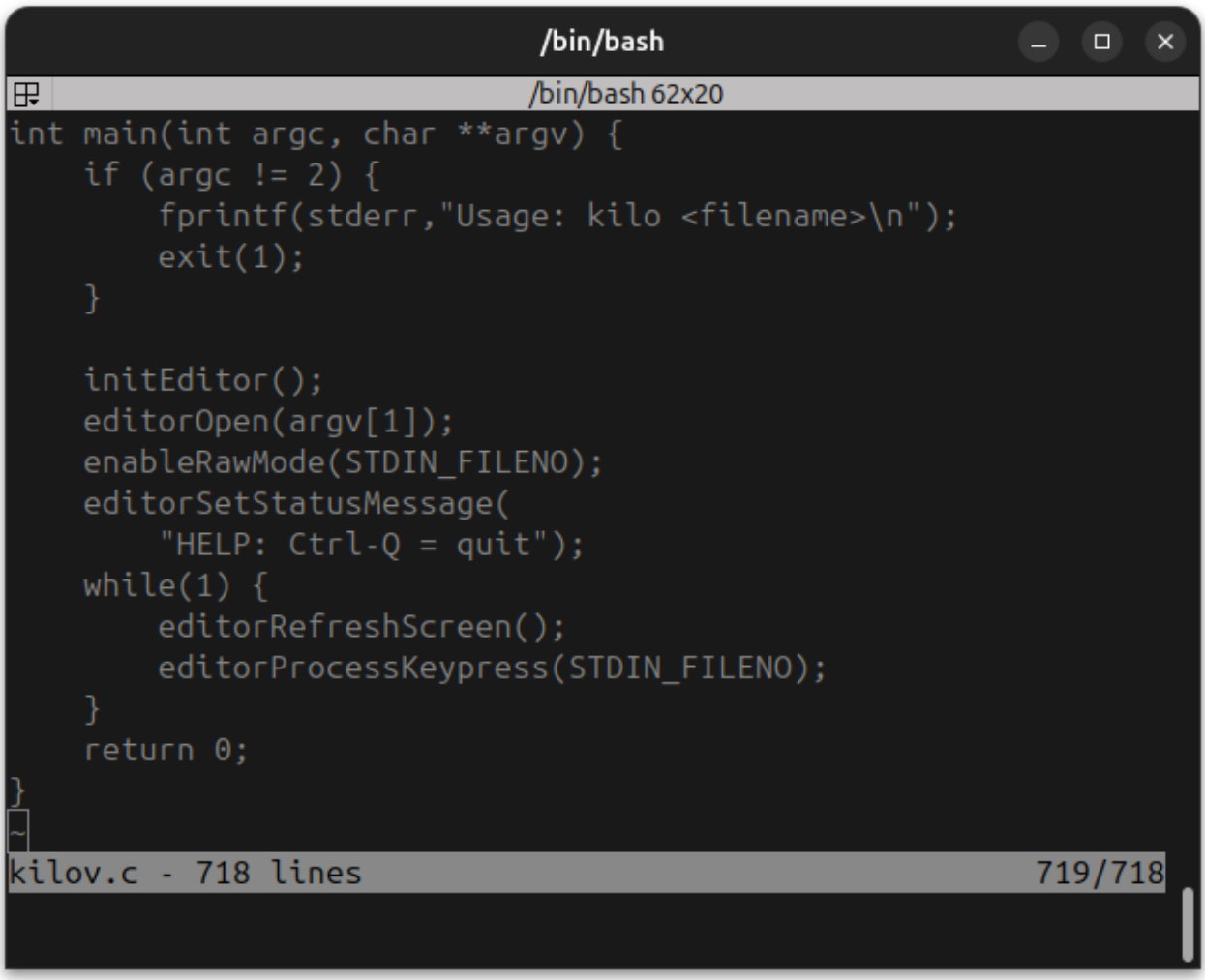


Figure 6: 課題4テスト6（最終行でEnterを押した後）

```
            exit(0);
            break;
        case PAGE_UP:
        case PAGE_DOWN:
            if (c == PAGE_UP && E.cy != 0)
                E.cy = 0;
            else if (c == PAGE_DOWN && E.cy != E.screenrows-1)
                E.cy = E.screenrows-1;
            {
            int times = E.screenrows;
            while(times--)
                editorMoveCursor(c == PAGE_UP ? ARROW_UP:
                                                ARROW_DOWN);
            }
            break;

        case ARROW_UP:
        case ARROW_DOWN:
        case ARROW_LEFT:
        case ARROW_RIGHT:
            editorMoveCursor(c);
            break;
+       case '-':
+           /* Move to the beginning of the previous line. */
+           editorMoveCursor(ARROW_UP);
+           E.cx = 0;
+           E.coloff = 0;
+           break;
+       case ENTER:
+           /* Move to the beginning of the next line. */
+           editorMoveCursor(ARROW_DOWN);
+           E.cx = 0;
+           E.coloff = 0;
+           break;
        case CTRL_L: /* ctrl+l, clear screen */
            /* Just refresht the line as side effect. */
            break;
        case ESC:
            /* Nothing to do for ESC in this mode. */
            break;
        default:
            break;
        }

        quit_times = KILO_QUIT_TIMES; /* Reset it to the original value. */
}
```

感想まとめ

今回の実験では、既存のプログラムを読み取り、その構造を理解したうえで新しい機能を追加する作業を行った。最初か

課題2では、キーボードから入力されるエスケープシーケンスと、プログラムから端末へ送るエスケープシーケンスがある
editorConfigがカーソル位置、スクロール位置、ファイル情報、端末状態をまとめて管理し、editorRefreshScreen()がその

課題4では、既存のeditorMoveCursor()を利用することで、上下移動や画面スクロールの処理を重複して書かずに機能を追

また、AI chatbotはプログラムの構造やエスケープシーケンスの調査、追加処理の確認に利用した。ただし、AIが出力し